



Portfolio Subproject 2

COURSE

COMPLEX IT SYSTEMS

Group 11 members:

IDA HAY JØRGENSEN	80088
JULIUS KRÜGER MADSEN	80734
MAREK LASLO	82428
SOFUS HILFLING NIELSEN	82278

Source Code Repository:

https://github.com/RUC-MSc-CS-CIT-2024/portfolio_subproject_2

November 6, 2024

Contents

1	Introduction	3
1.1	Application login/authentication	3
2	Application design	3
2.1	Application Layers	4
2.1.1	Data Layer	4
2.1.2	Business Layer	5
2.1.3	Web Service Layer:	5
3	Development	7
3.1	Structure of the objects	7
3.1.1	Domain Models	7
3.1.2	Data Transfer Objects	8
3.1.3	JSON objects	10
3.2	Authentication and Authorization	10
3.2.1	Token Generation	11
3.2.2	Authorization	12
3.3	REST API the Uniform Interface	12
3.3.1	HATEOAS	12
3.3.2	Pagination	13
3.4	Representation and Configuration	14
3.4.1	Data Annotations	14
3.4.2	Repository Pattern and Object-Relational Mapping	16
3.4.3	Example of Mapping Relationships	17
3.5	Domain Model	19
3.5.1	Key Entities	19
3.5.2	Relationships	19
3.5.3	Object-Relational Mapping	20
3.6	Database design changes	21
3.7	Testing	22
3.8	Class Diagrams	23

4	Reflections	23
4.1	Endpoint Security and Protection Against Malicious Clients	23
4.2	Scalability and memory concerns	24
4.3	Entity framework performance issues	24
5	Conclusion	25
6	Appendix	26

1 Introduction

This report builds upon the foundational work established in Portfolio 1, where core database structures and essential functionalities were implemented. The project is now extended with a backend developed as a Web API in ASP.NET. The primary features enable users to search, bookmark, and rate various types of media, while a centralized media model effectively links genres, cast, and user interactions across titles, including movies, series, and episodes.

1.1 Application login/authentication

Some endpoints have been protected with authorization and requires authentication to use. This is done by first generating a JSON Web Token through the `/api/auth/generate-token` POST endpoint, and then using the returned token for subsequent requests. The `generate-token` endpoint uses Basic authentication, see 3.2 for how to generate token with the default credentials¹. Endpoint requires HTTPS.

The JWT token is a Bearer token which is included in the Authorization header when querying other protected routes like so: `Bearer <token>`. When using Swagger (`/swagger/index.html`) this can be done in the top right corner through the green Authorize button.

2 Application design

This web application is designed following RESTful API principles to efficiently handle data transfer and translation between the movie database and the front-end client. With an emphasis on decoupling, it leverages the repository design pattern, HATEOAS (Hypermedia as the Engine of Application State), and a Model-View-Controller (MVC) structure to provide a scalable, maintainable, and flexible architecture.

The API is fully stateless, meaning each client request contains all necessary information, which enhances scalability and reduces server load. Each request is independent, allowing the server to efficiently manage multiple clients simultaneously. This setup

¹Username: john.doe, Password: password123

allows the client and server to evolve independently, using a standardized, predictable interface with HTTP methods like GET, POST, PATCH, and DELETE.

The repository pattern effectively separates business logic from data access, exposing a clear API for CRUD (Create, Read, Update, Delete) operations. Domain objects, representing core entities (e.g. Media, Genre), encapsulate the business data and behavior, providing a clear representation of the business model. Meanwhile, Data Transfer Objects, facilitate efficient and secure data exchange between the server and client.

Dependency injection is utilized to link components across layers, promoting loose coupling and facilitating unit testing. By relying on abstractions rather than specific implementations, each layer can be tested independently. This architectural choice enhances flexibility, making it simpler to implement changes or new features without disrupting existing functionality.

2.1 Application Layers

To better understand this architecture, the following sections detail each layer's functionality. The application is divided into three main layers. See figure 1.

2.1.1 Data Layer

The data layer manages the database connection and handles direct data manipulation.

Database Context: The first component of the data layer handles relational database mapping using Entity Framework Core with PostgreSQL integration. The project includes two distinct data context classes: one manages mappings specific to the movie database, while the other handles mappings related to the framework components of the movie database. This separation allows for clearer organization and management of database interactions.

Models and Database Entities: Domain models in the database context map relational database entities and their relationships. These models facilitate data transfer to the repository component and subsequently to the business layer.

Repository: The repository forms the second component of the data layer, implementing the repository pattern to store functions for direct data manipulation. Each repository class is responsible for a specific domain entity.

2.1.2 Business Layer

The business layer handles core business logic, including authentication and token creation, as well as any complex processing required by the application.

Manager: Manager classes are responsible for retrieving the required data from the data layer, transforming it into DTOs, and passing it to the API layer. This layer ensures that only essential, non-sensitive information is transferred to the next layer. Additionally, managers can enforce business rules, such as validating user input or applying business-specific logic before data is processed or stored.

Data Transfer Objects: In the business layer, DTOs encapsulate data transferred between business logic and the API layer.

2.1.3 Web Service Layer:

This layer manages HTTP methods and exposes endpoints to the client.

Controller: Controllers are classes that implement HTTP methods and define endpoints. They serve as the intermediary between the client and the business layer, receiving client requests and returning responses.

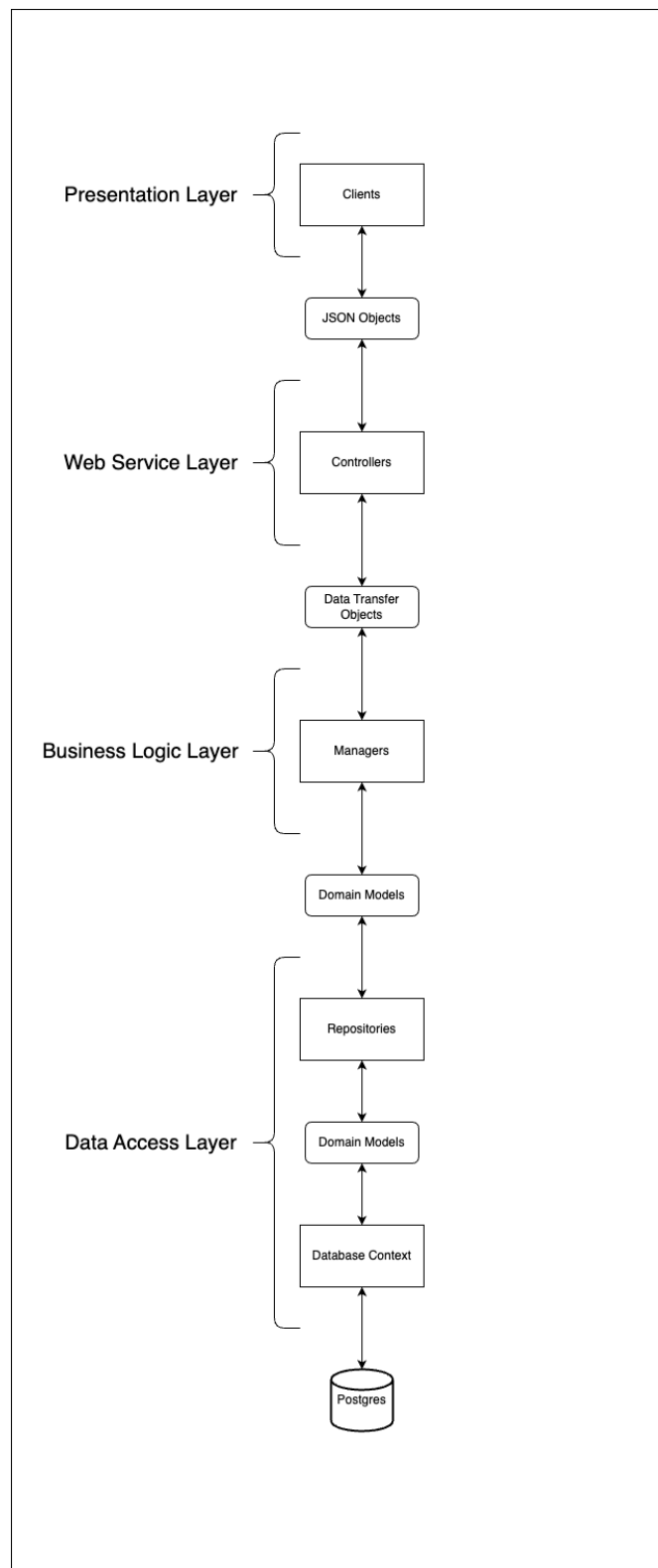


Figure 1: Back-end Architecture.

3 Development

This section details how the application was developed and the different challenges encountered during the development.

3.1 Structure of the objects

The structure of the objects within the application is as follows: domain objects, data transfer objects, and JSON objects.

3.1.1 Domain Models

Here is an example of the Domain Model for Media (code 1). The Media domain model represents the media entity. It encapsulates the logic and it is mapped to the database table. The attributes and their brief descriptions are:

- **Id:** Unique identifier for the media.
- **Type:** Type of media (e.g., Movie, TV Show).
- **Plot:** Plot summary of the media.
- **Runtime:** Duration of the media.
- **BoxOffice:** Box office earnings.
- **Budget:** Budget of the media.
- **ImdbId:** IMDB identifier.
- **Awards:** Awards won by the media.
- **MediaProductionCompany:** Associated production company.
- **Score, Release, Title, RelatedMedia, CastMembers, CrewMembers, Collections, Countries, Genres:** Collections representing various relationships with other entities.


```

[Table("media")]
public class Media
{
    [Key, Column("media_id")]
    public int Id { get; set; }

    [Column("type")]
    public required string Type { get; set; }

    [Column("plot")]
    public string? Plot { get; set; }

    [Column("runtime")]
    public int? Runtime { get; set; }

    [Column("box_office")]
    public int? BoxOffice { get; set; }

    [Column("budget")]
    public int? Budget { get; set; }

    [Column("imdb_id")]
    public string? ImdbId { get; set; }

    [Column("awards")]
    public string? Awards { get; set; }

    public MediaProductionCompany? MediaProductionCompany { get; set; }

    public ICollection<Score> Score { get; } = new List<Score>();
    public ICollection<Release> Release { get; } = new
    [...]
}

```

Code 1: Domain model for the media.

3.1.2 Data Transfer Objects

Here is an example of the DTO for transferring basic media information the server and the client (code 2). Objects like these are made to be small and only transfer the

necessary data for a specific operation. The attributes and their brief descriptions are:

- **Id:** Unique identifier for the media.
- **Type:** Type of media (e.g., Movie, TV Show).
- **Title:** Title of the media.
- **ReleaseDate:** Release date of the media.
- **PosterUri:** URI for the poster image.

```
public class MediaBasicResult
{
    public required int Id { get; set; }
    public required string Type { get; set; }
    public required string Title { get; set; }
    public DateTime? ReleaseDate { get; set; }
    public string? PosterUri { get; set; }
}
```

Code 2: DTO for MediaBasicResult.

The differences between the Media Domain Model and the MediaBasicResult DTO are the purpose of use and therefore their properties. MediaBasicResult DTO provides a structured format for packaging necessary information. Ensuring that only relevant information is transmitted. In DTOs, relationships with other entities are ignored and are only set in the domain model.

Pattern for Data Transfer Objects In the application, the DTOs follow a general pattern to ensure consistency and clarity. This pattern includes suffixes such as **Result**, **CreateRequest**, **UpdateRequest**, and **DeleteRequest** to denote the purpose of each DTO.

- **Result DTOs:** Used to transfer data from the server to the client.

- **CreateRequest DTOs:** Used to transfer data from the client to the server when creating a new resource.
- **UpdateRequest DTOs:** Used to transfer data from the client to the server when updating an existing resource.
- **DeleteRequest DTOs:** Used to transfer data from the client to the server when deleting an existing resource.

3.1.3 JSON objects

When receiving or sending results in response to requests, JSON objects serve as the external representation of the data. This format either transmits data from the API to a client or receives it from the client, giving clear and structured communication. Below is an example of the JSON object structure for the `MediaBasicResult` (code 3).

```
{
  "id": 1,
  "type": "Movie",
  "title": "Inception",
  "releaseDate": "2010-07-16T00:00:00Z",
  "posterUri": "http://example.com/inception.jpg"
}
```

Code 3: JSON object for MediaBasicResult.

3.2 Authentication and Authorization

The authentication flow consists of a login endpoint located at `/api/auth/generate-token`. The endpoint is protected with basic access authentication that requires the user to include an Authorization header within the format of `"Basic <credentials>"`. The credentials consist of a username and password separated by a colon and then base 64 encoded.

When a valid set of credentials is provided the endpoint generates a JSON Web Token (JWT) and returns it in its response body.

The JWT is used to access other resources in the system as well as protects specific endpoints from being accessed by anyone besides the resource owner e.g. endpoints that require the user id can only be accessed by a JWT with that user id.

3.2.1 Token Generation

The `JwtSecurityTokenHandler` class is used to generate a JWT

The token creation process is distinct from the validation of the token. A JWT consists of a header, payload and signature. The signature is a hash of the header and payload together with a 256-bit secret and the header contains information about how the token signature is generated. The application uses HS256 (HmacSha256) algorithm for the token generation.

The payload (code 4) of the token is a JSON object that contains information about the user who generated the token as well as when the token was generated, expires, etc. Each name/value pair in the payload is referred to as claims, a claim represents an assertion about the token subject. In the movie API it has been chosen to include both the `user_id` as well as their `email` this is used for validating who the user is as part of the authorization process when a token is used to access a protected resource. Other default claims are part of the token payload as well, like when it expires or the issuer of the token.

```
{
  "sub": "john_doe",
  "email": "john@example.com",
  "user_id": "1",
  "role": "default",
  "nbf": 1730237248,
  "exp": 1730238148,
  "iat": 1730237248,
  "iss": "CitMovie_Api",
  "aud": "CitMovie_Api"
}
```

Code 4: JWT JSON payload.

3.2.2 Authorization

The application uses ASP.NET's Authentication and Authorization middleware to handle incoming request validation and access to resources. The authentication middleware validates the JWT on the incoming request by using the signing key used to generate the JWT to verify its validity. The authorization middleware is used for verifying the access level of the request, this is done through a custom authorization policy that check whether the `user_id` in the JWT payload matches the `user_id` of the resource which the request is for.

3.3 REST API the Uniform Interface

As mentioned the application follows the REST (Representational State Transfer) API architectural pattern. REST defines a set of constraints that help guide API design and interaction, it must have:

1. Statelessness
2. Cacheability
3. Client-Server Separation
4. Layered System
5. Code on Demand (optional)
6. Uniform Interface

The uniform interface constrains four principles: resource identification, representation, self-descriptive messages, and HATEOAS (Hypermedia as the Engine of Application State).

3.3.1 HATEOAS

HATEOAS is an essential part of REST, enhancing the discoverability and usability of resources. HATEOAS is allowing for navigation between resources. This makes the API more flexible and self-explanatory for clients.

In the current implementation, HATEOAS dynamically generates URIs for navigating to specific resource pages. The **GetLink** method supports this by creating URIs for resource pages based on the current page number and page size. This is achieved in two steps:

1. `_linkGenerator.GetUriByName` generates the URI using the provided `linkName` and the route parameters (`page` and `pageSize`).
2. The method then returns the URI or null if the URI generation fails.

This dynamic linking allows clients to access paginated data seamlessly while navigating between pages.

3.3.2 Pagination

Pagination is essential for handling large data sets in REST APIs. This improves both client and server performance by limiting the data transferred at once. In this application, pagination is managed through query parameters:

- **page:** Defines the current page number.
- **pageSize:** Specifies the number of items displayed per page.

The **CreatePaging** method applies pagination in line with the HATEOAS principles by building a response object with these properties:

- **CurPage:** The URI of the current page, generated by calling `GetLink(linkName, page, pageSize)`.
- **NextPage:** The URI of the next page, if available; otherwise, null.
- **PrevPage:** The URI of the previous page, if available; otherwise, null.
- **NumberOfItems:** The total item count, providing the client with insight into the complete data scope.

- **NumberPages:** The total number of pages, determined by dividing the total item count by the pageSize and rounding up.
- **Items:** The actual data for the current page.

By calculating **numberOfPages** and dynamically generating navigation links (CurPage, NextPage, and PrevPage), the method gives the clients a clear pathways through the dataset, enabling efficient and seamless navigation. This design optimizes the user experience.

3.4 Representation and Configuration

During the development of the database and model layer, it was decided to use Data Annotations for configuring the models in Entity Framework. This approach provides a straightforward, embedded way to configure model properties directly within the model classes improving readability and maintainability.

3.4.1 Data Annotations

Data Annotations allow us to define key configurations such as primary keys, column names, and data types directly within the model class. This embedded configuration keeps the details close to the model logic making the model class self-contained. For example, using Data Annotations, we configure the Media and Person models as follows (Code 5):

```
namespace CitMovie.Models.DomainObjects;

[Table("media")]
public class Media
{
    [Key, Column("media_id")]
    public int MediaId { get; set; }

    [Column("type")]
    public string Type { get; set; }

    [Column("plot")]
    public string? Plot { get; set; }

    -- Rest of the properties are omitted --
}

[Table("person")]
public class Person
{
    [Key, Column("person_id")]
    public int PersonId { get; set; }

    [Column("name")]
    public string Name { get; set; }

    [Column("birth_date")]
    public DateTime BirthDate { get; set; }

    [Column("death_date")]
    public DateTime DeathDate { get; set; }

    -- Rest of the properties are omitted --
}
```

Code 5: Domain model objects used by Entity Framework.

In these examples, attributes such as [Table] and [Column] map properties in the model to columns in the PostgreSQL database, ensuring that EF Core correctly translates between the object-oriented model and relational schema.

3.4.2 Repository Pattern and Object-Relational Mapping

To create a flexible and maintainable database access layer, this implementation uses Entity Framework Core and the repository pattern to mediate database operations. The primary goals are to separate business logic from data access, support CRUD operations, and allow the Web Service Layer (WSL) or Business Layer (BL) to communicate effectively with the Data Access Layer (DAL).

Object-Relational Mapping Object-Relational Mapping (ORM) enables object-oriented languages to manage and interact with relational databases by converting data between incompatible systems. In this implementation, Entity Framework Core (EF Core) serves as the ORM to map the database schema to C# objects, encapsulating the complexities of raw SQL queries and allowing the development team to work with data in a more object-oriented way.

Framework Contexts and Models The application uses two `DbContext` classes (`FrameworkContext` and `DataContext`) to manage database access. The `FrameworkContext` is tailored to manage framework-specific entities (e.g., bookmarks, follows, user scores), while the `DataContext` addresses movie-related entities (e.g., media, genres, cast members). This separation ensures that each context handles only the data models within its scope, enhancing organization and facilitating maintainability. In both contexts, the `OnConfiguring` method configures EF Core to use a PostgreSQL database based on the provided connection string, ensuring database consistency across the application. The `OnModelCreating` method specifies additional configurations, including many-to-many relationships, composite keys, and specific mapping strategies, which align the relational database structure with object-oriented design.

Repository Pattern The repository pattern abstracts data access, making it possible to interact with database entities without directly dealing with SQL or EF Core queries. Each repository class is responsible for a specific domain entity, providing methods for CRUD operations while concealing the underlying database implementation. For example, `MediaRepository` handles operations for the `Media` entity, including custom

queries for searching, filtering, or fetching related entities. This structure promotes separation of concerns, allowing the business logic to operate independently of data access specifics.

Repositories leverage EF Core's LINQ capabilities for query generation, ensuring that data is efficiently retrieved and manipulated. By abstracting data access details, this design improves testability—mock repositories can replace actual ones in testing environments.

3.4.3 Example of Mapping Relationships

In object-relational mapping, relationships between entities mirror the connections in the relational model. Here are examples of how Entity Framework Core maps these relationships within the domain model:

One-to-One Relationship The `Media` and `MediaPrimaryInformation` entities exhibit a one-to-one relationship, where each media entity has a corresponding primary information entity. This is represented in the `Media` model by the `MediaPrimaryInformation` navigation property (Code 6):

```
public MediaPrimaryInformation PrimaryInformation { get; set; }
```

Code 6: One-to-One relationship in Media model.

In addition, `AutoMapper` is used to map the properties of `MediaPrimaryInformation` into Data Transfer Objects (DTOs) like `MediaBasicResult`. For instance, `AutoMapper` accesses `MediaPrimaryInformation` through the `PrimaryInformation` navigation property in `Media` to set DTO properties like `Title`, `ReleaseDate`, and `PosterUri`.

This separation of responsibilities ensures that the structural relationship is managed by EF Core while data mapping is handled by `AutoMapper`.

One-to-Many Relationship In the `User` entity, a one-to-many relationship with `UserScore` is defined, where each user can have multiple scores. This is represented in the `User`

model as an `ICollection<UserScore>`, while the `UserScore` model holds a reference to the `User` (Code 7):

```
public ICollection<UserScore> UserScores { get; set; }
```

Code 7: One-to-Many relationship in User model.

This mapping allows the system to navigate from a user to their scores and vice versa, supporting complex queries that retrieve related data in a single operation.

Many-to-Many Relationship The `Media` and `Genre` entities demonstrate a many-to-many relationship, where multiple media entries can have multiple genres, and vice versa. This relationship uses an implicit join table, created through the `UsingEntity` method in `OnModelCreating` (Code 8):

```
modelBuilder.Entity<Media>()
    .HasMany(e => e.Genres)
    .WithMany(e => e.Media)
    .UsingEntity("media_genre",
        l =>
            ↳ l.HasOne(typeof(Genre)).WithMany().HasForeignKey("genre_id"),
        r =>
            ↳ r.HasOne(typeof(Media)).WithMany().HasForeignKey("media_id"));
```

Code 8: Many-to-Many relationship between Media and Genre.

By configuring the `UsingEntity` method, EF Core manages the join table automatically, allowing efficient querying without manually handling the join table.

Interface for Repository Services Each repository defines an interface that standardizes CRUD operations, enabling the WSL or BL to interact with the database uniformly. For instance, `IMediaRepository` may include methods for common CRUD operations along with custom search methods to facilitate flexible querying by the business logic layer.

The abstract interface ensures that each layer adheres to the dependency inversion principle, promoting scalability and adaptability. For instance, adding new database sources or switching EF Core implementations is simplified, as only the repository implementation needs adjustment.

This object-relational mapping approach in Entity Framework Core, coupled with the repository pattern, achieves a clean separation of concerns and facilitates the development of a maintainable, testable, and scalable data access layer. This setup serves as a robust foundation for future feature enhancements and supports the application's core goal of handling complex media-related data with efficiency.

3.5 Domain Model

The domain model represents core entities and relationships within the application, illustrating how users interact with media content. This is only a snippet of the complete domain, and as such there will be entities not included for simplicity. This section provides an overview of some key entities, their attributes, and how relationships are mapped using Entity Framework.

3.5.1 Key Entities

- **User:** Represents an application user, with attributes such as *Id* and *Name*. Users can create bookmarks, complete media items, follow people, and maintain search histories. Each user is linked to multiple *UserScore*, *Bookmark*, *Completed*, *Follow*, and *SearchHistory* entries, capturing diverse interactions.
- **Media:** Represents media items (movies, series, etc.) and includes attributes like *Title*. Media entities are central, connecting to various entities like *Genre*, *ProductionCompany*, *UserScore*, *CastMember*, *CrewMember*, and *RelatedMedia*. This allows each media item to be associated with multiple genres, production companies, and user scores.

3.5.2 Relationships

One-to-One

- Each **Media** has one **MediaPrimaryInformation** entity, holding primary details like *Title* and *Release*.

One-to-Many

- **User** has one-to-many relationships with **UserScore**, **Bookmark**, **Completed**, **Follow**, and **SearchHistory**.
- **Media** has one-to-many relationships with **Release**, **UserScore**, **Bookmark**, **Completed**, **CastMember**, and **CrewMember**.

Many-to-Many

- **Media** has a many-to-many relationship with **Genre** and **ProductionCompany**, allowing multiple genres per media item and vice versa.

3.5.3 Object-Relational Mapping

Using EF, each entity is mapped to a database table, with attributes defining columns and relationships configuring foreign keys and join tables. The ORM automatically manages complex relationships:

- **One-to-One** and **One-to-Many** relationships use navigation properties (e.g., *MediaPrimaryInformation* in *Media*).
- **Many-to-Many** relationships are configured through intermediate join tables (e.g., *media_genre* for *Media* and *Genre*), enabling efficient database operations without explicit SQL.

This structured mapping with EF simplifies data handling by aligning the object-oriented domain model with the relational database, allowing seamless data retrieval and updates within the system. See (figure 2) for a visual representation of these key entities and relationships.

3.6 Database design changes

In the old version, to be able to fetch the primary release, primary title and primary promotional media of a media entity, it would be required to do multiple querying and filtering on 3 different tables. That data is crucial, to display the primary information of the media.

Roskilde University

information” was implemented into the database. This relation, is storing the primary release information of the media.(See fig 3)

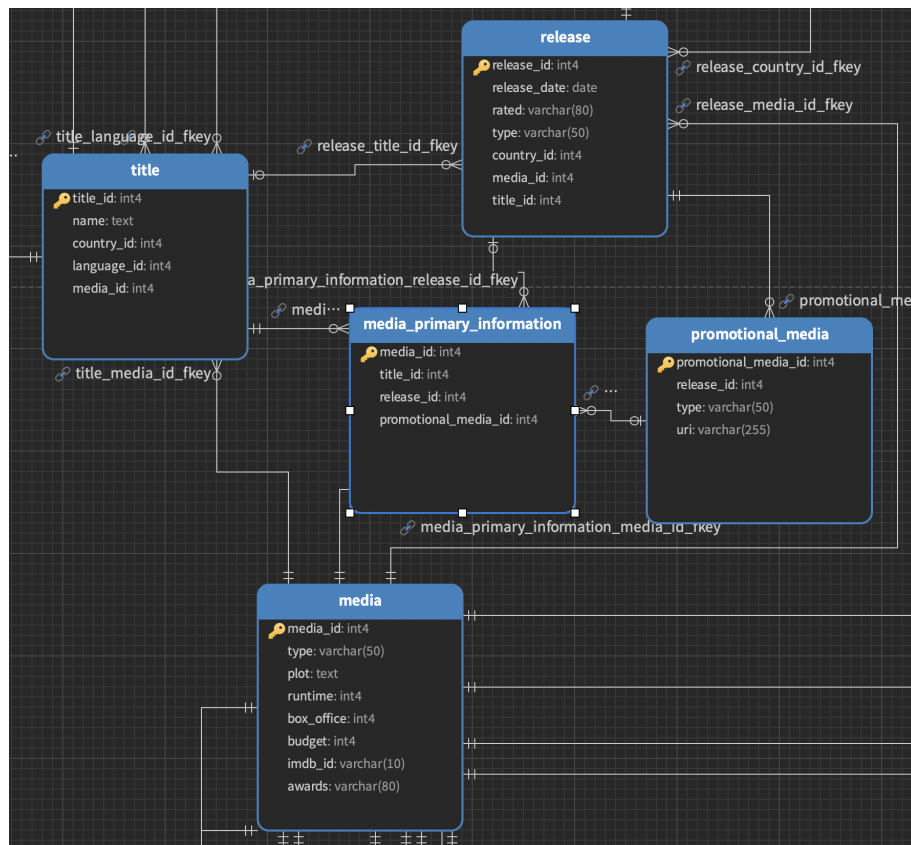


Figure 3: Promotional media information relation

3.7 Testing

Tests are implemented in the project to ensure that the different layers and their methods; function correctly and deliver data in the correct format. An example of this is the `PersonManagerUnitTest` class this unit test tests the service methods within `PersonManager`. Dependencies like `IPersonRepository` and `IMapper` are mocked using `FakeItEasy`, allowing the test to simulate behavior without using a real database.

In the test method `GetPersonsAsync_ValidParameters_CallsRepositoryAndMapsResult`, the goal is to validate the `GetPersonsAsync` method. The **arrange** section sets up the necessary data and configures the mocked repository and mapper behavior. In the

act section, `GetPersonsAsync` is called. The **assert section** then verifies that both the repository and mapper methods were called exactly once, ensuring that the result is correctly mapped to the expected DTO format. Tests like these are implemented to secure a preformed and working application.

3.8 Class Diagrams

At the end of a development process 5 class diagrams were generated.

An overall class diagram (See fig. 4 in Appendix), which describes the overall architecture of the solution.

Then there are Web Service Layer, Business Logic Layer, Data Access Layer and Model Class Diagram (See fig. 5, 6, 7, 8 in Appendix).

4 Reflections

In developing the API, it is essential to reflect on opportunities for improving performance, security, and usability. This section identifies specific areas within the application where optimizations could increase efficiency, reduce vulnerabilities, and enhance the user experience.

4.1 Endpoint Security and Protection Against Malicious Clients

One critical area of improvement for the current API is enhancing the security of the "create" endpoints. At present, these endpoints are exposed to the risk of misuse by malicious clients, who could potentially spam the server with repeated requests e.g. one could POST as many users as they want without restriction. This can overload the storage capacity and slow the response time causing a bad user experience. Some points to consider could be the use of **Rate limiting**, which would limit the number of requests a client can make in a certain period. **CAPTCHA** could also be a way to ensure that the requests are from a human. **Email verification** would also increase the credibility of the client that creates e.g. a user because they now need to verify their email address.

4.2 Scalability and memory concerns

The current implementation of `GetUserScoresAsync` retrieves user scores based on various filters, such as media type, media ID, and media name. However, this approach presents potential issues related to memory usage and scalability. Specifically, the method retrieves all matching media IDs and title media IDs, storing them in memory using `ToListAsync()`. This can result in high memory consumption when there are large numbers of entries that meet the filter criteria.

For example, suppose millions of media entries match a specific media type filter. In that case, loading all these IDs into memory simultaneously could lead to significant memory usage and may negatively impact the application's performance.

Optimizing this method is challenging due to the constraints of using two different contexts, but this issue will be investigated further.

4.3 Entity framework performance issues

A performance issue emerged in which the system froze when retrieving media objects, particularly when getting titles along with their types and attributes. This issue arises from using the Table-Per-Type (TPT) inheritance strategy in Entity Framework, a choice driven by the structure of the existing database design. The TPT configuration enables the creation of subclasses of the media type, such as `Episode` and `Season`. However, TPT significant performance penalty when compared to other inheritance strategies within EF. It performs more than twice as slowly compared to other models. Additionally, the application is affected by inefficient pagination (offset), and a recommended alternative approach is to use **keyset** pagination with a `WHERE` clause that filters the result.

5 Conclusion

In conclusion, this project builds upon the foundation established in Portfolio 1 by creating a backend in ASP.NET that integrates with a Web API. Designed with RESTful principles, a layered architecture, and security measures using JWT, this API enables user interactions with media content, it supports searching, bookmarking, and rating functionalities that were developed in portfolio 1 as functions for the PostgreSQL database. The application uses Entity Framework for data access, HATEOAS for navigation, and DTOs for data transfers, ensuring that the application will be easily scalable for future portfolio projects.

6 Appendix

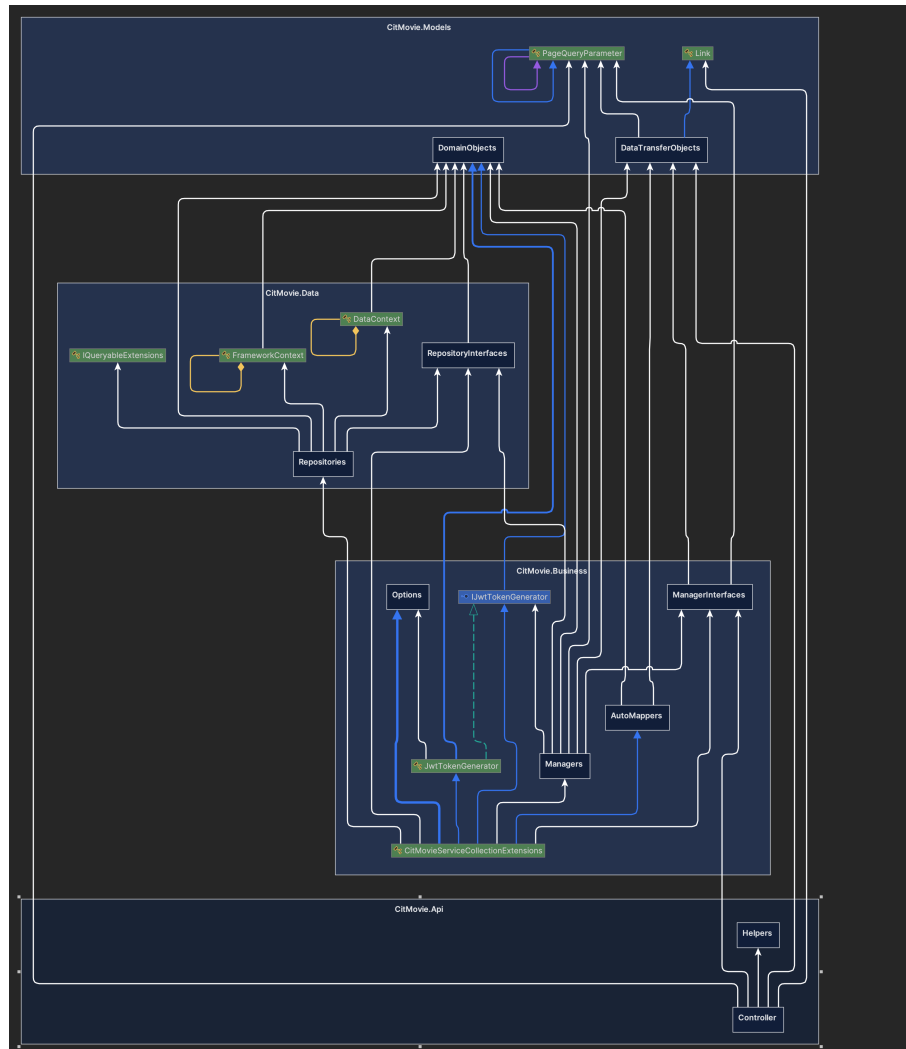


Figure 4: Overall Class Diagrams

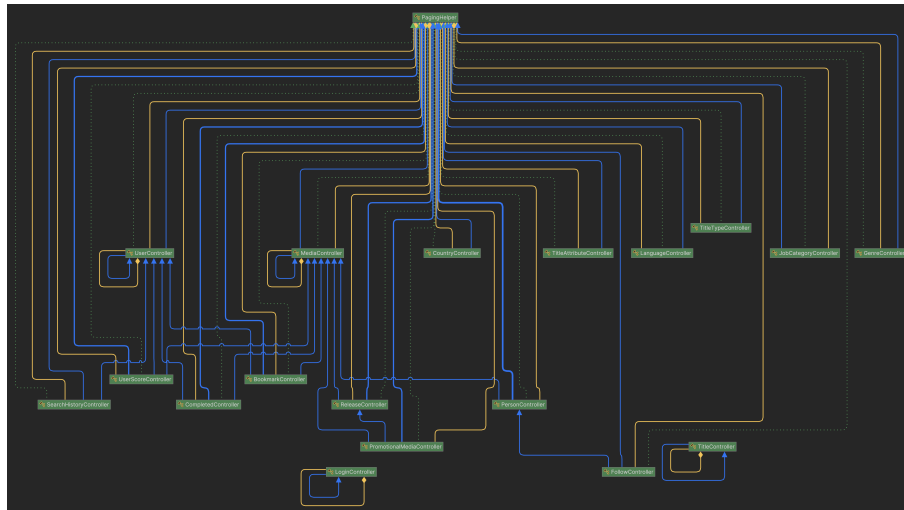


Figure 5: API Layer Class Diagram

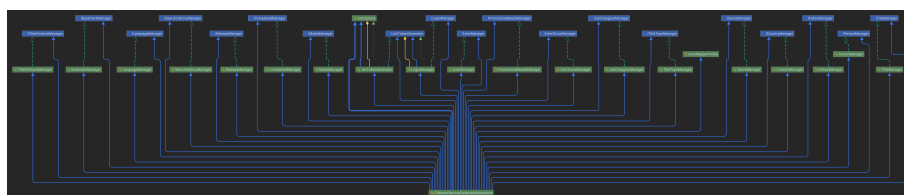


Figure 6: Business Layer Class Diagram

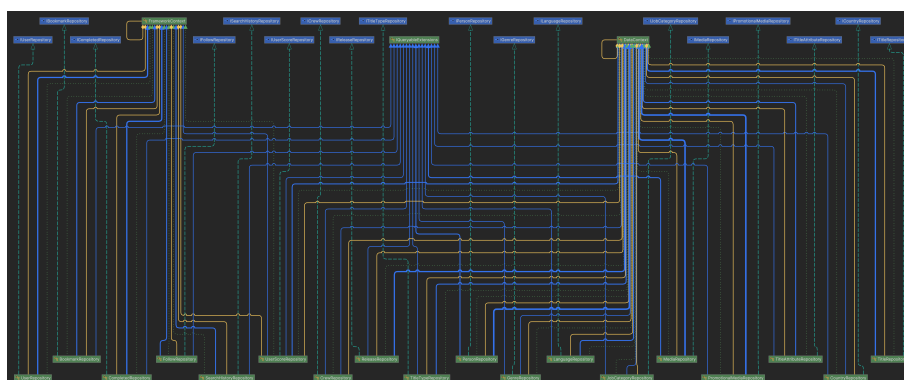


Figure 7: Data Layer Class Diagram

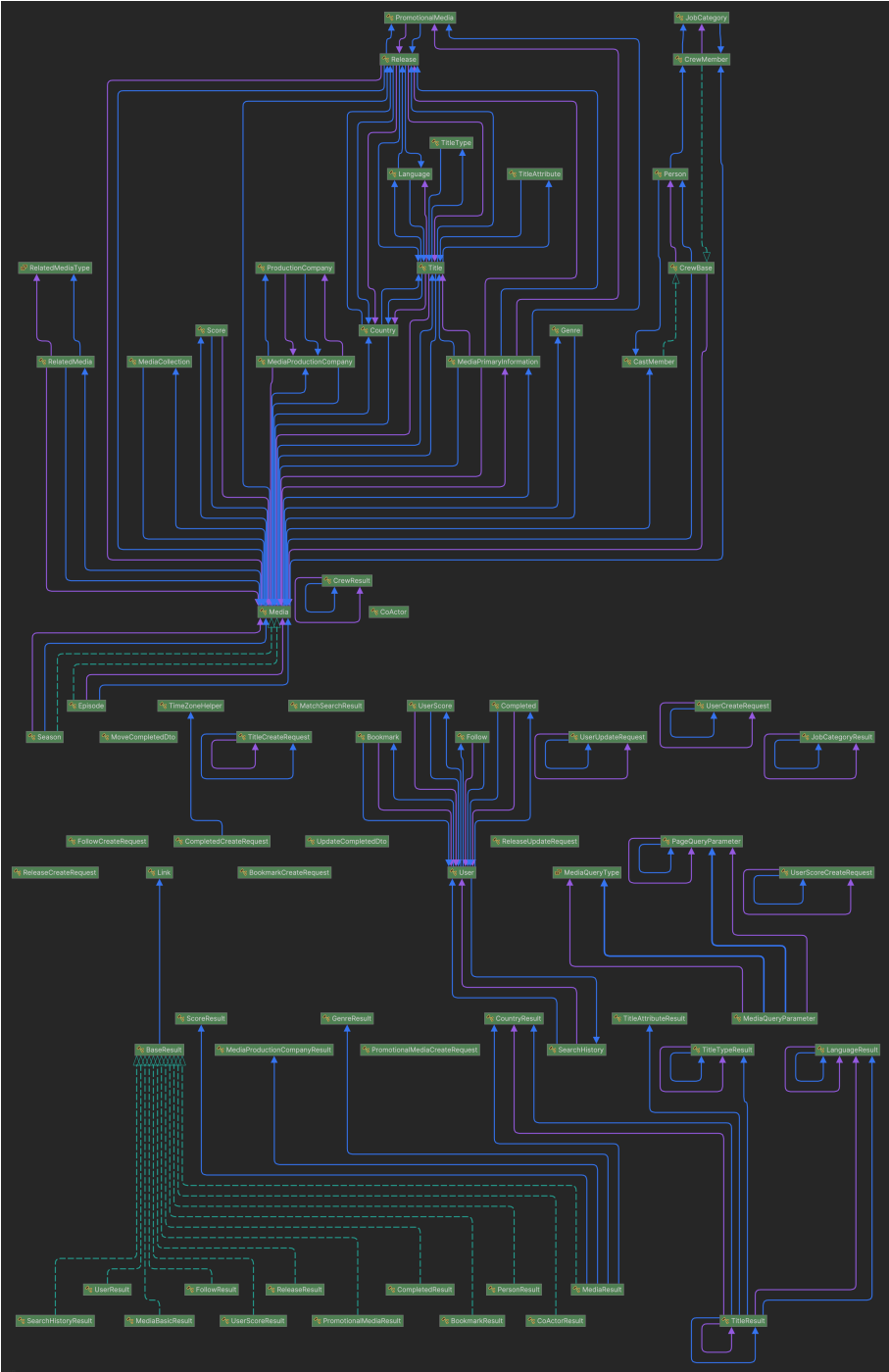


Figure 8: Model Class Diagram